

Workshop 2: Web Oriented Architecture

Lab Activity: Extending Web Applications with Event-Driven Architecture (EDA)

Application: University Room Booking System

Department of Computer Engineering
University of Peradeniya

1. Domain Event Identification

Application: University Room Booking System

Our application is a University Room Booking System that allows users to create, update, and delete room bookings for lectures and events. The system manages rooms with details (name, capacity, location) and bookings with time slots, preventing double-booking through overlap validation.

Selected API Operation

We selected the Create Booking operation (POST /api/bookings) as the trigger for our domain event. This is the most impactful operation in the system — when a new booking is successfully created, multiple downstream actions should occur (notifications, analytics, calendar sync, etc.).

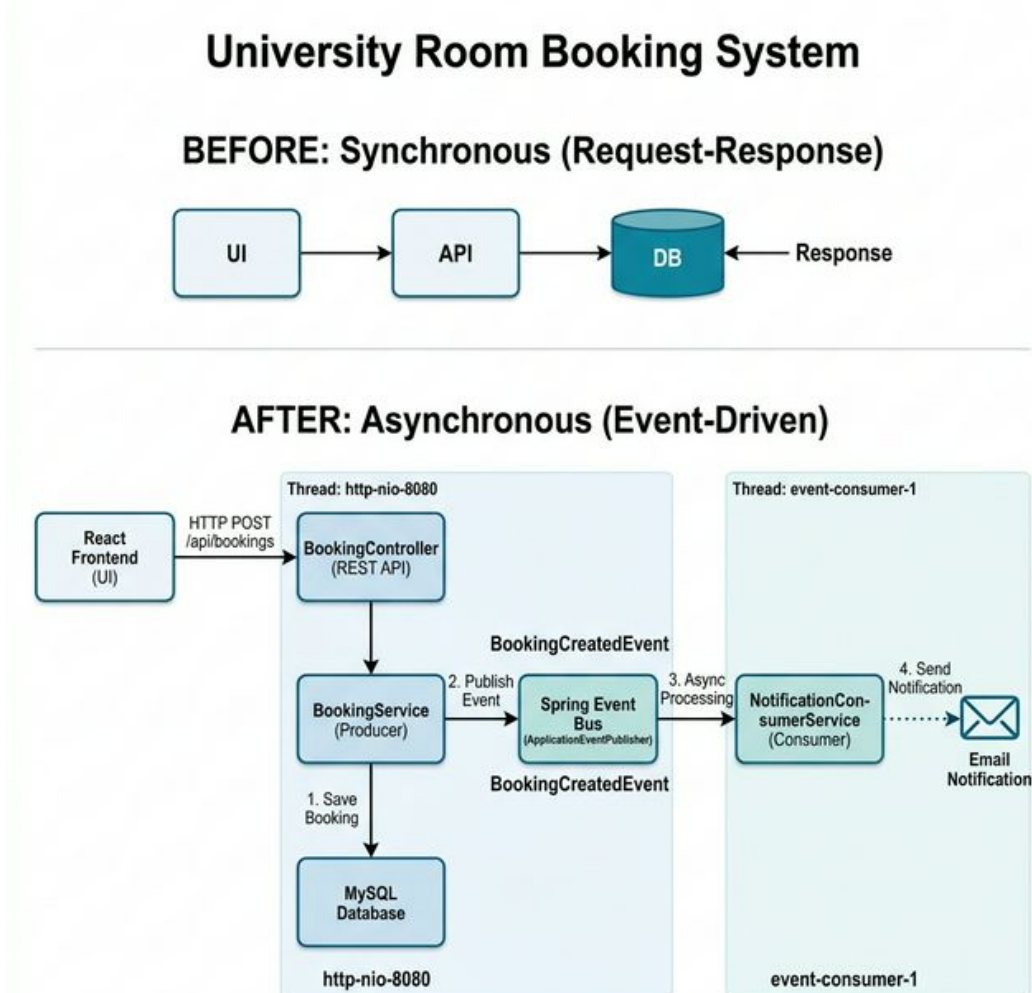
Domain Event: BookingCreatedEvent

The BookingCreatedEvent is published after a new booking is persisted to the database. It carries the following payload as required by the lab specification:

Field	Type	Description
eventName	String	BOOKING_CREATED
bookingId	Long	Entity ID of the created booking
roomName	String	Name of the booked room
bookedBy	String	User who made the booking
startTime / endTime	LocalDateTime	Time slot of the booking
timestamp	LocalDateTime	When the event was published

2. Architecture Diagram

The following diagram shows the transition from the original synchronous Request-Response model to the new asynchronous Event-Driven Architecture:



Key architectural components:

- Producer (BookingService) — publishes events after successful DB operations
- Event Bus (Spring ApplicationEventPublisher) — in-memory event routing
- Consumer (NotificationConsumerService) — processes events asynchronously on a separate thread pool

3. Event Producer Implementation

The BookingService was modified to act as the Event Producer. After saving a booking to the database, it publishes a BookingCreatedEvent to the Spring Event Bus. The API returns a response immediately without waiting for any downstream consumer processing.

BookingService.java

```
package com.university.booking.service;

import com.university.booking.entity.Booking;
import com.university.booking.entity.Room;
import com.university.booking.event.BookingCreatedEvent;
import com.university.booking.repository.BookingRepository;
import com.university.booking.repository.RoomRepository;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.ApplicationEventPublisher;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class BookingService {

    private static final Logger logger = LoggerFactory.getLogger(BookingService.class);

    @Autowired
    private BookingRepository bookingRepository;

    @Autowired
    private RoomRepository roomRepository;

    @Autowired
    private ApplicationEventPublisher eventPublisher;

    public Booking createBooking(Booking booking) {
        Room room = roomRepository.findById(booking.getRoom().getId())
            .orElseThrow(() -> new RuntimeException("Room not found"));
        booking.setRoom(room);

        List<Booking> overlappingBookings = bookingRepository
            .findByRoomIdAndEndTimeAfterAndStartTimeBefore(
                room.getId(), booking.getStartTime(), booking.getEndTime());

        if (!overlappingBookings.isEmpty()) {
            throw new RuntimeException("Room is already booked for the selected time slot.");
        }

        // Step 1: Database operation completes successfully
        Booking savedBooking = bookingRepository.save(booking);
        logger.info("[Producer] Thread: {} | Booking saved (ID: {})",
            Thread.currentThread().getName(), savedBooking.getId());

        // Step 2: Publish domain event AFTER DB commit
        BookingCreatedEvent event = new BookingCreatedEvent(
            savedBooking.getId(), room.getName(),
            savedBooking.getBookedBy(),
```

```
        savedBooking.getStartTime(), savedBooking.getEndTime());
    eventPublisher.publishEvent(event);
    logger.info("[Producer] BookingCreatedEvent published to Event Bus");

    // Step 3: API returns response immediately
    logger.info("[Producer] Returning response to client NOW");
    return savedBooking;
}
}
```

BookingCreatedEvent.java

```
package com.university.booking.event;

import java.time.LocalDateTime;

public class BookingCreatedEvent {

    private final String eventName;
    private final Long bookingId;
    private final String roomName;
    private final String bookedBy;
    private final LocalDateTime startTime;
    private final LocalDateTime endTime;
    private final LocalDateTime timestamp;

    public BookingCreatedEvent(Long bookingId, String roomName,
                               String bookedBy, LocalDateTime startTime,
                               LocalDateTime endTime) {
        this.eventName = "BOOKING_CREATED";
        this.bookingId = bookingId;
        this.roomName = roomName;
        this.bookedBy = bookedBy;
        this.startTime = startTime;
        this.endTime = endTime;
        this.timestamp = LocalDateTime.now();
    }

    // Getters, toString() ...
    public String getEventName() { return eventName; }
    public Long getBookingId() { return bookingId; }
    public String getRoomName() { return roomName; }
    public String getBookedBy() { return bookedBy; }
    public LocalDateTime getStartTime() { return startTime; }
    public LocalDateTime getEndTime() { return endTime; }
    public LocalDateTime getTimestamp() { return timestamp; }

    @Override
    public String toString() {
        return "BookingCreatedEvent{eventName='" + eventName +
            "', bookingId=" + bookingId +
            ", roomName='" + roomName +
            "', bookedBy='" + bookedBy +
            "', timestamp=" + timestamp + "}";
    }
}
```

4. Event Bus Configuration

We use Spring's built-in `ApplicationEventPublisher` as the in-memory Event Bus, combined with `@EnableAsync` and a custom `ThreadPoolTaskExecutor`. This ensures that event consumers run on a separate thread pool, providing true asynchronous behavior.

AsyncConfig.java

```
package com.university.booking.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.scheduling.annotation.EnableAsync;
import org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor;

import java.util.concurrent.Executor;

@Configuration
@EnableAsync
public class AsyncConfig {

    @Bean(name = "eventExecutor")
    public Executor eventExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(2);
        executor.setMaxPoolSize(5);
        executor.setQueueCapacity(25);
        executor.setThreadNamePrefix("event-consumer-");
        executor.initialize();
        return executor;
    }
}
```

5. Consumer Service Implementation

The NotificationConsumerService is an independent service that subscribes to BookingCreatedEvent. It runs asynchronously on a separate thread pool and simulates sending an email notification. A deliberate 2-second delay demonstrates that the consumer processes events independently of the main API thread.

NotificationConsumerService.java (Consumer)

```
package com.university.booking.consumer;

import com.university.booking.event.BookingCreatedEvent;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.event.EventListener;
import org.springframework.scheduling.annotation.Async;
import org.springframework.stereotype.Service;

@Service
public class NotificationConsumerService {

    private static final Logger logger =
        LoggerFactory.getLogger(NotificationConsumerService.class);

    @Async("eventExecutor")
    @EventListener
    public void handleBookingCreatedEvent(BookingCreatedEvent event) {
        logger.info("==== NOTIFICATION CONSUMER =====");
        logger.info("[Consumer] Thread: {} | Event: {}",
            Thread.currentThread().getName(), event.getEventName());
        logger.info("[Consumer] Booking ID: {} | Room: {} | By: {}",
            event.getBookingId(), event.getRoomName(),
            event.getBookedBy());
        logger.info("[Consumer] Sending email notification...");

        try {
            Thread.sleep(2000); // Simulate processing delay
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }

        logger.info("[Consumer] Notification sent for Booking ID: {}",
            event.getBookingId());
        logger.info("==== CONSUMER COMPLETE =====");
    }
}
```

6. Evidence of Functionality (Logs)

The following console output demonstrates the asynchronous event flow when a booking is created. Note the different thread names, confirming that the producer and consumer run on separate threads:

```
===== EVENT PRODUCER =====
[http-nio-8080-exec-1] BookingService : [Producer] Thread: http-nio-8080-exec-1 | Booking saved
successfully (ID: 5)
[http-nio-8080-exec-1] BookingService : [Producer] Thread: http-nio-8080-exec-1 | BookingCreatedEvent
published to Event Bus
[http-nio-8080-exec-1] BookingService : [Producer] Thread: http-nio-8080-exec-1 | Returning API
response to client NOW
===== PRODUCER COMPLETE =====

        ← API Response returned to client here

===== NOTIFICATION CONSUMER =====
[event-consumer-1] NotificationConsumerService : [Consumer] Thread: event-consumer-1 | Received event:
BOOKING_CREATED
[event-consumer-1] NotificationConsumerService : [Consumer] Booking ID: 5 | Room: Lecture Hall A |
Booked By: john@eng.pdn.ac.lk
[event-consumer-1] NotificationConsumerService : [Consumer] Time Slot: 2026-01-20T09:00 to 2026-01-
20T11:00
[event-consumer-1] NotificationConsumerService : [Consumer] Sending email notification to
john@eng.pdn.ac.lk...

        ← 2 second processing delay (simulating email send)

[event-consumer-1] NotificationConsumerService : [Consumer] Email notification sent successfully for
Booking ID: 5
===== CONSUMER PROCESSING COMPLETE =====
```

Key Observations:

- Producer runs on thread: http-nio-8080-exec-1 (main API thread)
- Consumer runs on thread: event-consumer-1 (separate async thread pool)
- API response is returned BEFORE the consumer finishes processing
- The 2-second consumer delay does NOT affect API response time

7. Decoupling Verification

To verify decoupling, we tested the system with the consumer service disabled. This was done by commenting out the `@Service` annotation on `NotificationConsumerService`, preventing Spring from registering it as a bean.

Test: Consumer Stopped, API Still Functions

With the consumer disabled:

1. The backend started successfully with no errors.
2. Creating a booking via `POST /api/bookings` returned HTTP 200 with the saved booking.
3. The event was published but no consumer was present to process it — the event was silently dropped.
4. `GET /api/bookings` returned all bookings correctly.

This proves that the producer and consumer are fully decoupled. The main API functionality is completely independent of whether the consumer is running.

Test Result:

Main API functions perfectly when consumer is stopped

No errors or exceptions in the producer when consumer is absent

Database operations complete normally

8. Technical Explanation

Our University Room Booking System was originally built using a synchronous Request-Response architecture where the REST API directly saved bookings to the MySQL database and returned the response. To evolve this into an Event-Driven Architecture, we identified the "BookingCreated" domain event, triggered whenever a new room booking is successfully persisted.

The BookingService acts as the Event Producer, publishing a BookingCreatedEvent to Spring's built-in ApplicationEventPublisher (our in-memory Event Bus) immediately after the database commit. The API response is returned to the client without waiting for any downstream processing.

The NotificationConsumerService acts as an independent Consumer, listening for BookingCreatedEvent using @Async and @EventListener annotations. It runs on a dedicated thread pool ("event-consumer-"), completely decoupled from the main API thread. This means if the consumer fails or is stopped, the booking API continues functioning normally.

This architecture provides three key benefits: loose coupling between services, improved resilience (API independence from consumers), and scalability through the ability to add new consumers without modifying the producer